Eggs

Linguistic examples with minimalist syntax

Version 0.7.0

May 2026

https://github.com/retroflexivity/typst-eggs

Eggs is a Typst package for typesetting linguistic examples. Its aim is to provide a Typst analogue to LaTeX `gb4e`, `expex`, `linguex`, etc. Additionally, it ships with `leipzig`-style gloss abbreviations.

This is a documentation on Eggs. It begins with a review of features, then discusses several tips and tricks, and closes with providing an extensive list of functions and their arguments.

1. Initialization

To use Eggs, first import it, and then set its config via a global show rule.[1]

```
#import "@preview/eggs:0.7.0"
#show: eggs
```

This same function is used to configure Eggs' settings. See Section 8.

2. Examples

The primary function of Eggs is `example`. It accepts any `content` and typesets it as a single top-level linguistic example.

```
The sentence below demonstrates the reading known as specificational.
#example[
  The primary function of Eggs is `example`.
]
The semantics of the DP to the left of _is_ has been an object of much debate.
```

The sentence below demonstrates the reading known as specificational.

(1) The primary function of Eggs is `example`.

The semantics of the DP to the left of *is* has been an object of much debate.

Examples are automatically numbered continuously, using a counter called "eggsample". The counter's first level numbers examples, the second numbers subexamples. To override automatic numbering, pass the `number` argument to an example or subexample. Examples with a custom number do not increment the counter, so numbering continues where it was left off.

```
The idea of Predicate Inversion stems from the seeming parallelism of the following
sentences.
#example[
  `example` is the primary function of Eggs.
]
#example(number: 1)[
  The primary function of Eggs is `example`.
]
```

The idea of Predicate Inversion stems from the seeming parallelism of the following sentences.

(2) `example` is the primary function of Eggs.

(1) The primary function of Eggs is `example`.

The counter can also be set to an absolute value with `#counter("eggsample").update(n)` or a relative one with `#counter("eggsample").update(it => it + n)`. Within an example, the

[1]This package follows Typst's tradition of not abbreviating function names too much. Naturally, to achieve more effortless (or TeX-like) experience, you can set some aliases on import, e.g.

```
#import "@preview/eggs:0.7.0": example as ex, subexample as subex, judge as j, ex-label as el, ex-ref as er
```

subexample counter can be set by passing a function that preserves the counter's first value, e.g. `#counter("eggsample").update(it => (it.at(0), n))` . Following examples will be numbered starting with the next integer.

By default, examples in each footnote are numbered separately. They also use their own formatting. The following footnote[2] demonstrates this.

3. Subexamples

Numbered lists inside examples (lines that begin with `+`) are automatically typeset as subexamples.

```
Compare the following sentences, of which only the former exhibits the Definiteness Effect.
#example[
  + There is a/\*the subexample.
  + Here is a/the subexample.
]
```

Compare the following sentences, of which only the former exhibits the Definiteness Effect.

(3) a. There is a/*the subexample.
b. Here is a/the subexample.

In case you prefer it manual, the function `subexample` is defined. It is intended to only be used inside an example. It behaves pretty similarly to `example` (e.g. accepts `number`).

Automatic conversion of numbered lists can be toggled off by setting `auto-subexamples: false` in the config (see Section 8). To suspend it for a single example, pass `auto-subexamples: false` to the example directly.

```
Compare the following sentences, of which only the former exhibits the Definiteness Effect.
#example(auto-subexamples: false)[
  #subexample[There is a/\*the subexample.]
  #subexample[Here is a/the subexample.]
]
```

Compare the following sentences, of which only the former exhibits the Definiteness Effect.

(3) a. There is a/*the subexample.
b. Here is a/the subexample.

[2]Examples in footnotes use a separate counter, `fn-eggsample`, which is set to 0 at the beginning of each footnote.

```
But note the contrast in availability of indefinites.

#example[
  `example` is the/a function of Eggs.
]
#example[
  The/??a function of Eggs is `example`.
]
```

But note the contrast in availability of indefinites.

(i) `example` is the/a function of Eggs.

(ii) The/??a function of Eggs is `example`.

3.1. Customizing how subexamples are displayed

By default, automatic subexamples are simply placed one after another. This can be changed by passing or setting `subexample-wrapper`. A wrapper is a function that accepts any number of `content` elements and returns `content` constructed from them.

A simple and common use case is to align subexamples horizontally using a grid.

```
Unfortunately, according to the conference guidelines, in this abstract, our space is
limited to two pages. Consequently, to be able to fit everything included in our agenda, we
will try to save as much space as possible by aligning subexamples horizontally instead of,
how it is usually done in linguistic papers, vertically.

#show: eggs.with(subexample-wrapper: grid.with(columns: 5))

#example[
  + This is also a subexample.
  + This, too, is a subexample.
  + This is a subexample, too.
]
```

Unfortunately, according to the conference guidelines, in this abstract, our space is limited to two pages. Consequently, to be able to fit everything included in our agenda, we will try to save as much space as possible by aligning subexamples horizontally instead of, how it is usually done in linguistic papers, vertically.

(4) a. This is also a subexample. b. This, too, is a subexample. c. This is a subexample, too.

4. Glosses

Bullet lists in examples (lines that begin with `- `) are automatically treated as gloss lines.

For words to split, you need to ensure that there is a `space` element between them. The easiest way to do it is to separate words with **more than one** space. There are exceptions with function calls, so use ~ (non-breaking space) for spaces you don't want to be treated as separators.

Translations and preambles are written as lines below and above the list, respectively.

```
The following example presents the Russian _eto_-construction.
#example[
  Russian
  - Jajca      eto   vkusno.
  - eggs  this            tasty
  'Eggs are tasty.'
]
```

The following example presents the Russian *eto*-construction.

(5) Russian
Jajca eto vkusno.
eggs this tasty
'Eggs are tasty.'

Glosses can by typeset manually with `gloss`. It accepts either a content, which it splits automatically, or a list. Automatic bullet list conversion can be toggled off by setting `auto-glosses:`

`false` in the config (see Section 8). To suspend it for a single example, pass `auto-glosses: false` to the (sub)example directly.

```
The following example presents the Russian _eto_-construction.
#example(auto-glosses: false)[
  Russian
  #gloss(
    [Jajca      eto   vkusno.],
    ([eggs], [this], [tasty]),
  )
  'Eggs are tasty.'
]
```

The following example presents the Russian *eto*-construction.

(5) Russian
Jajca eto vkusno.
eggs this tasty
'Eggs are tasty.'

5. Judges

Certain strings at the beginning of a line or a gloss word (see Section 4) are automatically transformed into left judges.[3] Left judges are negatively padded strings that take up no space.

By default, such strings are `*`, `#`, `?`, `OK`, and all combinations of these. All except the asterisk are also superscripted. Spaces after a judge are omitted for readability.

```
The following pair shows the information-structural rigidity of specificational sentences.
#example[
  + OK SMITH is the killer.
  +  \*THE KILLER is Smith.
]
```

The following pair shows the information-structural rigidity of specificational sentences.

(6) a. ^{OK}SMITH is the killer.
b. *THE KILLER is Smith.

This can be modified by tweaking `auto-judges` in the config (see Section 8) or by passing the `auto-judges` argument to an example directly. `auto-judges` takes a dictionary where keys are the strings to treat as judges and values indicate whether to additionally superscript them. For instance, if you would like to make hashes and question marks non-superscripted, try `auto-judges: ("\*": false, "\#": false, "?": false, "OK": true)`. Use `auto-judges: ()` to disable the automatic conversion completely.

Again, a function `judge` is provided to typeset judges manually. Following spaces are **not** omitted.

[3] In fact, due to the way show rules work, this happens at the beginning of any elements inside examples, including inline ones. This might overgenerate left judges in certain narrow cases like in `this one _?here_`. Disable auto judges if it is a problem.

```
The following pair shows the information-structural rigidity of specificational sentences.
#example(auto-judges: ())[
  + #judge(super[OK])SMITH is the killer.
  + #judge[\*]THE KILLER is Smith.
]
```

The following pair shows the information-structural rigidity of specificational sentences.

(5) a. ^{OK}SMITH is the killer.
b. *THE KILLER is Smith.

N.B. Avoid using `judge` with strings that are already in `auto-judges`, as this can lead to superscript doubling.

6. Abbreviations

Eggs provides the whole set of Leipzig Standard Abbreviations as commands, in the style of TeX's `leipzig`. They are imported from a subpackage `abbreviations`.

All abbreviations are in lowercase. All abbreviations' names are as they appear, except for:

- **1, 2, 3** are `p1`, `p2`, `p3`
- **N** (as the non- prefix) is `non`
- **TOP** is `top_`, due to conflicting with Typst's built-in `top` alignment
- **INT** is `int_`, due to conflicting with Typst's built-in `int` type

```
#import abbreviations: ins, pst, n

Note that the famous temperature examples forbid Instrumental in Russian.
#example[
  - \*Temperatur-oj  byl-o        10~gradusov.
  - weather-#ins             be-#pst\-#n  10~degrees
]
```

Note that the famous temperature examples forbid Instrumental in Russian.

(6) *Temperatur-oj byl-o 10 gradusov.
weather-INS be-PST-N 10 degrees

Custom abbreviations can be created by defining a new `abbreviation`.

```
#let eto = abbreviation("eto", "an extremely polysemous expression")

Due to the polysemy _eto_ exhibits, I will gloss it simply as #eto.
#example[
  - Eto   što   za   primer?
  - #eto  what  for  example
  'What's this example?'
]
```

Due to the polysemy *eto* exhibits, I will gloss it simply as ETO.

(7) Eto što za primer?
ETO what for example
'What's this example?'

The list of currently used abbreviations is stored as a dictionary of the form `abbr: description` inside the `used-abbreviations` state. The final list of abbreviations can be accessed as `#context state("used-abbreviations").final()`. There is also a function to print it as a term list.

```
The list of glossing abbreviations used in this paper:
#print-abbreviations()
```

The list of glossing abbreviations used in this paper:
3 third person
AI animate intransitive
AN animate
DUR durative
ETO an extremely polysemous expression
INS instrumental
N neuter
PL plural
PRO pronoun
PST past

7. Labels and refs

There are two ways of labelling an example (or a subexample). The first is passing the `label` argument to `example` (or `subexample`). The second is placing an `ex-label` anywhere in the body of the example (or a subexample), preferably at the beginning or at the end. Typst's built-in `label` does not work for now.

If a subexample doesn't have a label but its parent does, an automatic codly-style label of the form `<top-label:subex-letter>` is added automatically.

```
The curious minimal pair in terms of scope in @pair is taken from Arregi et al. (2021).
Only @pred has a narrow scope reading of the indefinite.
#example[
  #ex-label(<pair>)
  + OK Kim is not one of the judges. #ex-label(<pred>)
  + \#One of the judges is not Kim.
]
@pair:b is pragmatically odd, but becomes better if Kim is replaced with something that can
exist in multiple places at once @ok. The argument loses in convincibility, though.
#example(label: <ok>)[
  OK One of the judges is not "OK".
]
```

The curious minimal pair in terms of scope in (8) is taken from Arregi et al. (2021). Only (8a) has a narrow scope reading of the indefinite.

(8) a. OK Kim is not one of the judges.
b. # One of the judges is not Kim.

(8b) is pragmatically odd, but becomes better if Kim is replaced with something that can exist in multiple places at once (9). The argument loses in convincibility, though.

(9) OK One of the judges is not "OK".

References to examples are automatically enclosed in parentheses. Citing two examples together is possible by inserting the second reference as the supplement, in square brackets after the first one. When the second reference is to a subexample, the superexample number is collapsed.

```
Recall the discussion concerning examples @pair[@ok]. Despite the Russian data, we should
not ignore the minimal pair in @pred[@pair:b].
```

Recall the discussion concerning examples (8-9). Despite the Russian data, we should not ignore the minimal pair in (8a-b).

A dedicated function, `ex-ref`, is also provided. It accepts from 1 to 2 references, but also

- integers for relative numbering, with `0` referring to the last example, `1` to the next, etc.
- `left` and `right` positional arguments for content to appear in the parenthesis before and after example numbers.

```
Despite the Russian data, we should not ignore the minimal pair in #ex-ref(<pred>,
<pair:b>). There is much more to the story #ex-ref(left: "e.g. ", 1, right: " etc.").

#example[
  + Only "?" is one of the judges.
  + One of the judges is only "?".
]
```

Despite the Russian data, we should not ignore the minimal pair in (8a-b). There is much more to the story (e.g. 10 etc.).

(10) a. Only "?" is one of the judges.
b. One of the judges is only "?".

`smart-refs: false` can be passed to the configuration to disable automatic reference decoration.

8. Customization

Eggs offers some layout and styling customization and several behaviour options. The complete list is given in Section 10.

There are several ways of customizing the package options. The primary one is passing arguments to `eggs` in a global show rule.

```
#show: eggs.with(
  indent: 2.5em,
  body-indent: 2.5em,
  sub-indent: -0.3em,
  sub-body-indent: 1.7em
)

The examples in this part all imitate Higgins' (1973) original layout.
#example[
  + What John also is is enviable.
  + What John also is is also enviable.
]

#example[
  What I am pointing at is a kangaroo.
]
```

The examples in this part all imitate Higgins' (1973) original layout.

(11) a. What John also is is enviable.
b. What John also is is also enviable.

(12) What I am pointing at is a kangaroo.

To change Eggs' config temporarily, you can scope the show rule or simply pass some content to `eggs`. Passed parameters are applied on top of the old configuration, so any parameters set earlier that are not overridden are preserved.

```
#eggs(indent: 2.5em, body-indent: 2.5em, sub-indent: -0.3em, sub-body-indent: 1.7em)[
  The following examples imitate Higgins' (1973) original layout.
  #example[
    + What John also is is enviable.
    + What John also is is also enviable.
  ]
  #example[
    What I am pointing at is a kangaroo.
  ]
] // this ends the scope of eggs
Compare #ex-ref(0) with itself but in the default layout.
#counter("eggsample").update(it => it - 1)
#example[
  What I am pointing at is a kangaroo.
]
```

The following examples imitate Higgins' (1973) original layout.

(11) a. What John also is is enviable.
b. What John also is is also enviable.

(12) What I am pointing at is a kangaroo.

Compare (12) with itself but in the default layout.

(12) What I am pointing at is a kangaroo.

To override the configuration for a single example, you can also pass the options to it directly. However, note that options are split among Eggs' functions: if you want to customize subexample or gloss options, you have to pass those to the functions directly (without prepending `sub-` or `gloss-`).

```
The following example imitates Higgins' (1973) original layout.
#example(indent: 2.5em, body-indent: 2.5em)[
  #subexample(indent: -0.3em, body-indent: 1.7em)[What John also is is enviable.]
]
```

The following example imitates Higgins' (1973) original layout.

(11) a. What John also is is enviable.

Finally, since Eggs is powered by Elembic, you may apply its set rules to the elements `example`, `subexample`, and `gloss`. Elembic's show rules can also be used to style example contents in an arbitrary way.

9. How-to's

Eggs strives to stay simple, in the sense that it does not convolute the syntax with non-Typst constructs, and does not add marginal functionality that is easy to implement on one's own.

Below is a partly community-maintained list of tips that might be useful for typesetting examples, but are not part of Eggs' core functionality.

9.1. Align content between subexamples

Say you want to include some phonological processes, with arrows aligned. An easy way is to define a function that draws a single-line grid, then place it in every subexample.

```
// provide some default width for the left column
#let phono-grid(ur, sr, ur-width: 4em) = grid(
  columns: (ur-width, auto, auto),
  gutter: 1em,
  ur, $->$, sr
)

#example[
  // override the width if UR doesn't fit
  #let phono-grid = phono-grid.with(ur-width: 5em)
  + #phono-grid([/mi-te-iru/], [[miteiru]])
  + #phono-grid([/kiri-te-iru/], [[kitteiru]])
]
```

(12) a. /mi-te-iru/ → [miteiru]
b. /kiri-te-iru/ → [kitteiru]

A more sophisticated function can include splitting the content automatically and measuring the width of an element. You can then use it as a subexample wrapper (see Section 3).

```
#import "@preview/elembic:1.1.1" as e
// accept a list of subexamples
#let phono-grid-wrapper(..args) = {
  // get the first and the last children of every row, insert arrow in between
  let lines-split = args.pos().map(it => {
    // get the elembic element's body
    let children = e.fields(it).body.children
    (children.at(0), $->$, children.at(-1))
  })
  // align the first column by the widest UR
  let ur-width = calc.max(..lines-split.map(it => measure(it.at(0)).width))
  for line-split in lines-split {
    // reassemble the subexample
    subexample(grid(
      columns: (ur-width, auto, auto),
      gutter: 1em,
      ..line-split
    ))
  }
}

#example(subexample-wrapper: phono-grid-wrapper)[
  + #[/mi-te-iru/] #[[miteiru]]
  + #[/kiri-te-iru/] #[[kitteiru]]
]
```

(13) a. /mi-te-iru/ → [miteiru]
b. /kiri-te-iru/ → [kitteiru]

9.2. Hide elements and pad under brackets in glosses

When typesetting glosses, it's common that you need to put a bracket next to the following word with no tabulation between them, but align the other lines with the **word**, not the bracket.[4]

A clean way to do this is to add a hidden printed bracket with the built-in hide.

```
#let hb() = hide[\[]
#example[
  - cet   exemple  a    [un     DP]
  - this  example  has  #hb()a  DP
]
```

(14) cet exemple a [un DP]
this example has a DP

In other cases, you may want to skip a word in some gloss line that is present in others. `hide` is good, but a simple non-breaking space (~) also suffices.

```
#example[
  - où     est  passé  le   mot   _t_  aujourd'hui
  - where  is   moved  the  word   ~   today
]
```

(15) où est passé le mot *t* aujourd'hui
where is moved the word today

9.3. Float attributions and other text right

Conventionally in linguistic examples, attributions are typeset as right-aligned text on the same line as the rest of the gloss. Adding `1fr` of horizontal spacing (100% of the available width) before an attribution will right-align it.

If the line is too long and the attribution doesn't fit, insert a linebreak before it.

```
#example[
  + Jones is a Jones.#h(1fr) (Burge 1973)
  + This entity called 'John J. Jones Jr.' is necessarily an entity called 'John J. Jones
Jr.'.\ #h(1fr) (Burge 1973, adapted to make the line even longer)
]
```

(16) a. Jones is a Jones. (Burge 1973)
b. This entity called 'John J. Jones Jr.' is necessarily an entity called 'John J. Jones Jr.'.
(Burge 1973, adapted to make the line even longer)

N.B. This method does not work on gloss lines.

9.4. Number examples by chapter

Package headcount provides graceful numbering dependent on current chapter. However, due to a bug, we need to tweak the definition of its `dependent-numbering` a bit.

[4] `expex` uses `@` and `\nogloss` for this.

Unfortunately, using `ex-ref` (and smart refs) with headcount is currently broken with cross-chapter references. This will probably be fixed when the next release of Elembic is dropped. For now, use @-refs with `smart-refs` off.

```
#import "@preview/elembic:1.1.1" as e
#import "@preview/headcount:0.1.0": *

// tweak the definition
#let dependent-numbering(style, levels: 1) = (..ns) => numbering(style, ..normalize-
length(counter(heading).get(), levels), ..ns.pos())

#show: eggs.with(
  smart-refs: false,
  auto-labels: false,
  // pass `level` if needed
  num-pattern: dependent-numbering("(1.1)"),
  ref-pattern: dependent-numbering("1.1a"),
  body-indent: 3em,
)
#show heading: reset-counter(counter("eggsample"))

= Suppose we have a chapter here

#example[
  and an example in the chapter #ex-label(<dep-counted>)
]

We can refer to it like this #ex-ref(<dep-counted>) and like this #ex-ref(0).

= And another here

We should refer to that example like this (@dep-counted).
```

---

1. Suppose we have a chapter here

(1.1) and an example in the chapter

We can refer to it like this (1.1) and like this (1.1).

2. And another here

We should refer to that example like this (1.1).

9.5. Avoid escaping hyphens after abbreviations by defining a glossary

Typst recognizes hyphens (-) as part of a variable name. This requires one to escape hyphens that mark morpheme boundaries if they immediately follow a variable reference, which can be annoying.

It can be convenient to instead define *aliases* for common morphemes that include the hyphen marker directly. This also has the advantage of working well with autocomplete if you have a large inventory of lexical entries.

```
#import abbreviations: p3, an, dur, pl, pro
#let ai  = abbreviation("ai",  "animate intransitive")

#let ann    = [that]
#let iksi   = [-#p3#pl.#an]
#let ponoka = [elk]
#let a_dur  = [#dur\-]
#let otsi   = [swim.#ai]
#let yaawa  = [-#p3#pl=#pro]
#let ohkan  = [#smallcaps[all]-]

#example[
  + - ann-iksi   ponoka-iksi   á-otsi-yi=aawa
    - #ann#iksi  #ponoka#iksi  #a_dur#otsi#yaawa
    'Those elk are swimming.'

  + - ann-iksi   ponoka-iksi   á-ohkan-otsi-yi=aawa
    - #ann#iksi  #ponoka#iksi  #a_dur#ohkan#otsi#yaawa
    'Those elk are all swimming.'
]
```

---

(1) a. ann-iksi ponoka-iksi á-otsi-yi=aawa
that-3PL.AN elk-3PL.AN DUR-swim.AI-3PL=PRO
'Those elk are swimming.'

b. ann-iksi ponoka-iksi á-ohkan-otsi-yi=aawa
that-3PL.AN elk-3PL.AN DUR-ALL-swim.AI-3PL=PRO
'Those elk are all swimming.'

9.6. Add line notes within examples with term lists

Eggs overrides auto-numbered lists (+) and bulleted lists (-), so you can't use them to add notes in examples. However, term lists remain untouched. These are particularly useful when typesetting notes prefixed with the speaker's initials.

The look of term lists can also be modified using show rules.

```
#import "@preview/elembic:1.1.1" as e
// style term lists inside examples only, with elembic
#show: e.show_(example, it => {
  // show the term and the description separated by a semicolon
  show terms.item: it => [#it.term: #it.description\ ]
  it
})

#example[
  + - anníksi  ponokáíksi̥  #highlight[á]ótsiyaaw̥ą̥.
  / EY: When you talk about swimming, _á-_ is like "being in the moment of"
  / EY: The difference here is between "swimming" vs. "swim"
  / EY: 'Those elk swim' is a statement of fact... doesn't imply any particular time
]
```

---

(2) a. anníksi ponokáíksi̥ áótsiyaaw̥ą̥.
EY: When you talk about swimming, *á-* is like "being in the moment of"
EY: The difference here is between "swimming" vs. "swim"
EY: 'Those elk swim' is a statement of fact... doesn't imply any particular time

10. Complete function documentation

10.1. eggs

Update the config with provided values, as well as run some necessary preparations. Primarily intended for use in a global show rule: `#show: eggs()`

- `it` `[content]`: The content. -> content
- `auto-subexamples` `[bool]`: Whether to treat numbered lists in examples as subexamples.
  Default: true
- `auto-glosses` `[bool]`: Whether to treat bullet lists in examples as glosses.
  Default: true
- `auto-labels` `[bool]`: Whether to insert subexample labels of the form ex-label:a.
  Default: true
- `auto-judges` `[dictionary]`: A dictionary of characters to convert into judges (keys) and whether to superscript them (values).
  Default: ("*": false, "#": true, "?": true, "OK": true)
- `subexample-wrapper` `[function]`: A function to wrap the subexample list. Should accept any number of arguments and return content. E.g. to align your subexamples horizontally, pass `grid.with(columns: 5)`. Only works for automatic examples.
  Default: (..args) => {args.pos().join()}
- `indent` `[length]`: Distance between the left margin and the left edge of the example number.
  Default: 0em
- `body-indent` `[length]`: Distance between the left edge of the example marker and the left edge of the example body.
  Default: 2.5em
- `sub-indent` `[length]`: Distance between the left edge of the top-level example body and the left edge of the subexample number. Can be negative.
  Default: 0em
- `sub-body-indent` `[length]`: Distance between the left edge of the subexample marker and the subexample body.
  Default: 1.5em
- `spacing` `[length]`: Vertical spacing around example. Currently, there is no way to modify spacing between two examples specifically.
  Default: current `par.spacing`.
- `sub-spacing` `[length]`: Vertical spacing around subexamples.
  Default: current `par.leading`.
- `breakable` `[bool]`: Whether the example figure is breakable.
  Default: false
- `sub-breakable` `[bool]`: Whether the subexample figure is breakable.
  Default: false

- `num-pattern` `[str | function]`: Example number format. A numbering pattern.
  Default: “(1)”
- `footnote-num-pattern` `[str | function]`: Example number format inside footnotes. A numbering pattern.
  Default: “(i)”
- `sub-num-pattern` `[str | function]`: Subexample number format. A numbering pattern.
  Default: “a.”
- `smart-refs` `[bool]`: Whether to format `@`-references and `ref`-references to examples Adding parenthesis and parsing the supplement.
  Default: true
- `ref-pattern` `[str | function]`: Example reference format. A 2-level numbering pattern.
  Default: “1a”
- `second-sub-ref-pattern` `[str | function]`: Format to reference the second argument of `ex-ref` if it is a subexample. A 1-level numbering pattern.
  Default: “a”
- `footnote-separate-numbering` `[bool]`: Whether examples in each footnote start from 1.
  Default: true
- `footnote-ref-pattern` `[str | function]`: Footnote example reference format. A 2-level numbering pattern.
  Default: “ia”
- `label-supplement` `[str | none]`: The example figure supplement used in references. Has no effect when `smart-ref` is `true`.
  Default: none
- `sub-label-supplement` `[str | none]`: The subexample figure supplement used in references. Has no effect when `smart-ref` is `true`.
  Default: none
- `gloss-word-spacing` `[length]`: Horizontal spacing between words in glosses.
  Default: 1em
- `gloss-line-spacing` `[length]`: Vertical spacing between lines in glosses.
  Default: current `par.leading`.
- `gloss-before-spacing` `[length]`: Vertical spacing above glosses (i.e. after the preamble).
  Default: current `par.leading`.
- `gloss-after-spacing` `[length]`: Vertical spacing below glosses (i.e. before the translation).
  Default: current `par.leading`.
- `gloss-styles` `[array]`: List of functions to be applied to each line of glosses. Can be of any length. `gloss-styles[0]` is applied to the first line, `gloss-styles[1]` — to the second, etc. E.g. `(emph, it => it + [.])` makes the first line italicized and adds a period to the second line.
  Default: ()

10.2. subexample `content`

Second-level linguistic subexample. Only intended for use inside an example.

- `body` `[content]`: The body of the subexample.
  Required
- `number` `[int | none]`: Overrides automatic numbering of the subexample. If not none, the counter does not increment.
  Default: none
- `auto-glosses` `[bool]`: Whether to treat bullet lists as glosses.
  Default: true
- `auto-judges` `[dictionary]`: A dictionary of characters to convert into judges (keys) and whether to superscript them (values).
  Default: ("*": false, "#": true, "?": true, "OK": true)
- `indent` `[length]`: Distance between the left edge of the top-level example and the left edge of the subexample number.
  Default: 0em
- `body-indent` `[length]`: Distance between the left edge of the subexample marker and the left edge of the subexample body.
  Default: 1.5em
- `spacing` `[length]`: Vertical spacing around the subexample.
  Default: current `par.leading`.
- `breakable` `[bool]`: Whether the subexample figure is breakable.
  Default: false
- `num-pattern` `[str | function]`: Subexample number format. A numbering pattern.
  Default: "a."
- `ref-pattern` `[str | function]`: Example reference format. A 2-level numbering pattern.
  Default: "1a"
- `second-sub-ref-pattern` `[str | function]`: Format to reference the second argument of `ex-ref` if it is a subexample. A 1-level numbering pattern.
  Default: "a"
- `label-supplement` `[str | none]`: The subexample figure supplement used in references. Has no effect when `smart-ref` is `true`.
  Default: none

10.3. example `content`

Top-level linguistic example.

- `body` `[content]`: The body of the example.
  Required

- `number` `[int | none]`: Overrides automatic numbering of the example. If not none, the counter does not increment.
  Default: none
- `auto-subexamples` `[bool]`: Whether to treat numbered lists in examples as subexamples.
  Default: true
- `auto-glosses` `[bool]`: Whether to treat bullet lists in examples as glosses.
  Default: true
- `auto-labels` `[bool]`: Whether to insert subexample labels of the form ex-label:a.
  Default: true
- `auto-judges` `[dictionary]`: A dictionary of characters to convert into judges (keys) and whether to superscript them (values).
  Default: ("*": false, "#": true, "?": true, "OK": true)
- `indent` `[length]`: Distance between the left margin and the left edge of the example number.
  Default: 0em
- `body-indent` `[length]`: Distance between the left edge of the example marker and the left edge of the example body.
  Default: 2.5em
- `spacing` `[length]`: Vertical spacing around the example. Currently, there is no way to modify spacing between two examples specifically.
  Default: current `par.spacing`.
- `breakable` `[bool]`: Whether the example figure is breakable.
  Default: false
- `num-pattern` `[str | function]`: Example number format. A numbering pattern.
  Default: "(1)"
- `ref-pattern` `[str | function]`: Example reference format. A 2-level numbering pattern.
  Default: "1a"
- `label-supplement` `[str | none]`: The example figure supplement used in references. Has no effect when `smart-ref` is `true`.
  Default: none
- `smart-refs` `[bool]`: Whether to format `@`-references and `ref`-references to examples Adding parenthesis and parsing the supplement.
  Default: true

10.4. gloss `content`

Interlinear gloss grid.

- `body` `[content]`: Any number of rows of equal length. Rows can be either contents where elements are separated by more than one space or lists.
  Required

- `word-spacing` `[length]`: Horizontal spacing between words in glosses.
  Default: 1em
- `line-spacing` `[length]`: Vertical spacing between lines in glosses.
  Default: current `par.leading`.
- `before-spacing` `[length]`: Vertical spacing above glosses (i.e. after the preamble).
  Default: current `par.leading`.
- `after-spacing` `[length]`: Vertical spacing below glosses (i.e. before the translation).
  Default: current `par.leading`.
- `styles` `[array]`: List of functions to be applied to each line of glosses. Can be of any length. `gloss-styles[0]` is applied to the first line, `gloss-styles[1]` — to the second, etc. E.g. `(emph, it => it + [.])` makes the first line italicized and adds a period to the second line.
  Default: ()

10.5. judge

Typesets the content, then adds negative space of the length of this content. Used to add a judge to the left of a text.

- `j` `[content]`: The content to typeset
  Required

10.6. ex-label

Used inside an example to give it a label. Effectively an alias of metadata.

- `l` `[label]`: The label.
  Required

10.7. ex-ref

Typesets an example reference in parentheses.

- `..args` `[label | int]`: One to two arguments. Can be labels or integers (or elements, but you probably won't need it). Integers are used for relative reference: 0 means the last example, 1 means the next, etc.
  Required
- `left` `[content]`: Text on the left, e.g. "e.g. "
  Default: none
- `right` `[content]`: Text on the right, e.g. " etc."
  Default: none

10.8. abbreviation

Prints the symbol in smallcaps and adds it to the list of used abbreviations.

- `symbol` `[str]`: The abbreviation
  Required

- `description` `[str]`: The abbreviation description for the list of used abbreviations.
  Required

10.9. print-abbreviations

Prints the list of abbreviations used in the document as a term list.

- `sorted-by` `[function]`: A callback function to sort abbreviations by. Takes in a left and a right abbreviation and returns a boolean. To sort by order of use, pass `(_, _) => true`.
  Default: Alphabetical. `(l, r) => l <= r`